

Aula 1: Bases da Programação

Luís Felipe

UFF

18 de Agosto de 2025

Objetivos do Curso

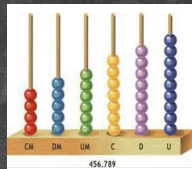
- Capacitar o(a) aluno(a) a compreender e aplicar os fundamentos da programação procedural, com foco na linguagem Python.
- Desenvolver a habilidade de:
 - ▶ **Representar** algoritmos com clareza e lógica.
 - ▶ **Resolver** problemas computacionais.
 - ▶ **Utilizar** estruturas de controle, variáveis, funções e estruturas de dados.
- Ao final da disciplina, espera-se que o(a) aluno(a) seja capaz de:
 - ▶ **Projetar** algoritmos para problemas diversos;
 - ▶ **Implementar** em Python, com boas práticas de codificação;
 - ▶ **Manipular** listas, matrizes, strings, tuplas e dicionários;
 - ▶ **Utilizar técnicas** de busca, ordenação e recursividade;
 - ▶ Trabalhar com entrada e saída de **arquivos**;
 - ▶ **Testar e depurar** seus programas com autonomia.
- O curso enfatiza também a construção de soluções, o raciocínio lógico, a organização modular dos programas e a clareza no código.

Por que estudar programação?

- Se você não é de computação, então por que programar?
 - ▶ Porque é legal.
 - ▶ Posso ter esse diferencial na minha carreira.
 - ▶ Posso criar programas para gerenciar e/ou automatizar algum processo.
 - ▶ Posso desenvolver novas ferramentas ou protótipos.
 - ▶ Posso criar e gerir simuladores.
 - ▶ Mesmo que você não implemente os algoritmos, será capaz de elaborá-los e interagir com o pessoal de TI.
- Para Matemática, Física, Química e outras ciências:
 - ▶ Como cientistas, vocês devem propor uma hipótese e testá-la (Aplicar o Método Científico).
 - ▶ Pelo **método científico**, você deve fazer:
 - ▶ Observação;
 - ▶ Problematização;
 - ▶ Hipótese;
 - ▶ Experimentação;
 - ▶ Análise dos resultados;
 - ▶ Conclusão;
 - ▶ Comunicação dos resultados

Um pouco de história

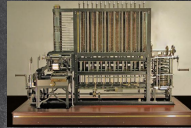
- 2400 AC: Ábaco
 - ▶ Primeira tentativa de se criar um artefato capaz de contar.
 - ▶ Representa números no sistema decimal e conseguimos trabalhar operações aritméticas com ele.
- 1642: Pascalina
 - ▶ Criada por Blaise Pascal, aos 19 anos.
 - ▶ Uma das primeiras máquinas mecânicas de calcular



Um pouco de história

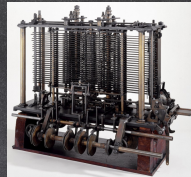
- 1822: Máquina Diferencial - Projetada por Charles Babbage.

- ▶ Primeiro computador mecânico, calcula automaticamente tabelas matemáticas, especialmente funções polinomiais.



- 1837: Máquina analítica - Projetada por Charles Babbage.

- ▶ Com conceitos modernos de computação como memória, unidade de controle e até leitura de cartões perfurados.
- ▶ Nela, já havia **instruções condicionais** e **laços de repetição**.



- ▶ Obs.: A matemática **Ada Lovelace** colaborou com Babbage, escrevendo anotações e algoritmos para a Máquina Analítica, por isso é considerada a primeira programadora da história.

Um pouco de história (continuação)

- 1936: Máquina de Turing - Proposta por Alan Turing, matemático britânico.

- ▶ Modelo teórico de um computador universal capaz de simular qualquer algoritmo computacional.
- ▶ Marco fundamental para a ciência da computação: nasce a ideia de que "**programa**" e "**dados**" são tratáveis formalmente.



- Relevância histórica:
 - ▶ Enquanto Babbage projetou uma máquina física com componentes semelhantes aos dos computadores modernos, Turing propôs um **modelo lógico e matemático** da computação.
 - ▶ Sua máquina influenciou a criação dos primeiros computadores eletrônicos e o campo da computação teórica.

Um pouco de história (continuação)

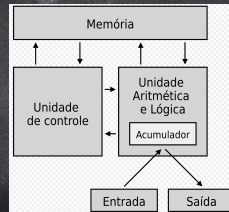
- 1941: Z3 de Konrad Zuse - Desenvolvido por **Konrad Zuse**.

- ▶ Primeiro computador eletromecânico programável da história.



- 1945: Arquitetura de von Neumann - Proposta por **John von Neumann**.

- ▶ Organização de computadores com memória única para dados e instruções, usado até hoje.



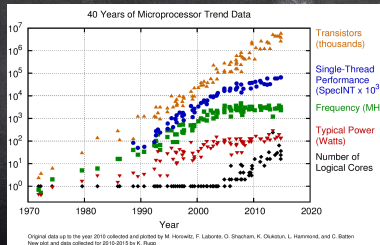
Mais um pouco de história (continuação)

ENIAC vs. EDVAC

- 1946: ENIAC - primeiro computador eletrônico de uso geral.
 - ▶ Cálculos balísticos.
 - ▶ Programação física (fiação manual).
 - ▶ Rápido, mas rígido.
- 1949: EDVAC - introduz o conceito de programa armazenado.
 - ▶ Arquitetura lógica mais próxima da atual.
 - ▶ Projeto com participação de John von Neumann.
 - ▶ Dados e instruções na mesma memória.
- Essa evolução permite a criação de linguagens de programação.

Lei de Moore

- Proposta por **Gordon Moore** em 1965.
- Observou que o número de transistores em um chip dobrava a cada 18–24 meses.
- Transistor: Interruptor eletrônico minúsculo, que pode estar em estado ligado (1) ou desligado (0). Ele é a base da lógica binária.
- Isso implicava em:
 - ▶ Mais poder de processamento
 - ▶ Menor custo por chip
 - ▶ Dispositivos mais compactos

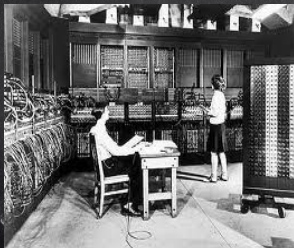
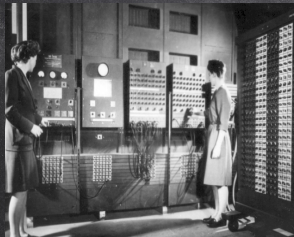


Avanços recentes na computação

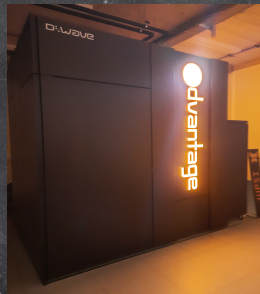
- 1970–1980: Microprocessadores e Computadores Pessoais
 - ▶ Popularização do uso doméstico e educacional dos computadores.
 - ▶ Apple, IBM, Microsoft iniciam revolução da computação pessoal.
- 1990–2000: Internet e a era da informação
 - ▶ Conectividade global transforma a forma como vivemos, estudamos e trabalhamos.
 - ▶ Surgimento da Web, navegadores e redes sociais.
- 2010–Atualidade: Computação em Nuvem, IA, Dispositivos Inteligentes e Computação Quântica
 - ▶ Avanços em **inteligência artificial**, **machine learning** e **automação**.
 - ▶ Popularização de assistentes virtuais, IoT, smartphones e computação em nuvem.
 - ▶ Início da era da **computação quântica**, com experimentos em máquinas capazes de resolver problemas além dos limites dos computadores clássicos.

Luís Felipe
18/08/25

“Avanço” da computação



ENIAC - 1946



D-Wave - 2022

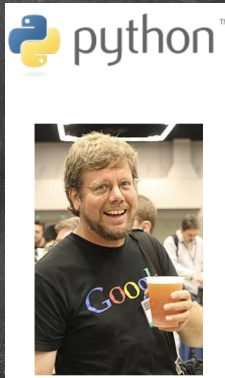
Da máquina ao código: Linguagens de programação

- Conforme os computadores evoluíram, tornou-se necessário um meio de comunicação entre humanos e máquinas.
- No início, as instruções eram dadas diretamente em linguagem de máquina (binário).
- Com o tempo, surgiram linguagens de programação:
 - ▶ **Assembly** (baixo nível, anos 1950)
 - ▶ **Fortran, COBOL** (alto nível, anos 1960)
 - ▶ **C, Pascal** (anos 1970–80)
 - ▶ **Java, Python, Ruby** (anos 1990–2000)
- Essas linguagens permitem escrever programas de forma mais próxima da lógica humana, sem se preocupar com os detalhes eletrônicos do hardware.

Python

Python foi criado e idealizado na década de 80 e sua implementação começou em 1989, por Guido van Rossum.

- 2000: Python 2
- 2008: Python 3



Conceitos básicos

Computador: Máquina que pode executar programas

Algoritmo: Sequência lógica e bem definida de passos para resolver um problema

Programa: Implementação de um algoritmo em uma linguagem de programação

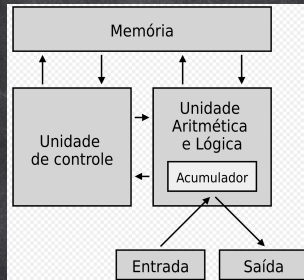
Linguagem de programação: Linguagem projetada para produzir programas de computadores

Hardware

Hardware: Dispositivos físicos que compõem um computador.

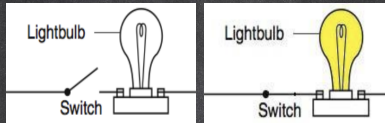
- Exemplo: CPU, Disco Rígido, Memória, etc.

Estes dispositivos seguem uma organização básica da Arquitetura de Von Neumann.



Hardware

Todo o hardware opera com sinais digitais: sem energia e com energia. Normalmente usamos valores 0 e 1 para representar isto.



- Chamamos estes sinais de **Bit** → valores 0 ou 1.
- Chamamos de **Byte** → um agrupamento de 8 bits.
Ex.: $123 = 1 + 2 + 8 + 16 + 32 + 64 = 01111011$.
- Todas as informações armazenadas no computador são representadas por números 0s e 1s. Informações como letras, símbolos, imagens e programas são todas codificadas como sequências de 0s e 1s.
 - Microprocessador:
 - Move o conteúdo de grupos de bits
 - Soma pares de grupos de bits
 - Subtrai um grupo de bits de outro
 - Compara pares de grupos de bits

Software

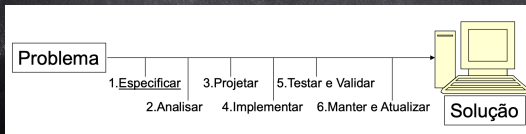
Software: Programas que executam tarefas utilizando o hardware de um computador.

- Os softwares são compostos por um conjunto de instruções que operam o hardware.
- Temos abaixo, por exemplo, três instruções para um computador de 32 bits.

```
0100 0010  0011 0101  0101 0100  0011 0110
0100 1110  1100 1100  1001 0110  0110 1000
0000 0101  1111 1110  1101 0011  0000 1100
```

- Um software é composto por milhares de instruções deste tipo.

Ciclo de vida de um programa:



Organização básica de um ambiente computacional

Um ambiente computacional é organizado como uma hierarquia de funções, onde cada uma é responsável por uma tarefa específica.

Ambiente de aplicação → Compiladores → Sistema Operacional → Hardware

Ambiente Computacional: Como usuários, interagimos com os programas de aplicação. Neste curso iremos descer nesta hierarquia, para **construirmos novos programas de aplicação**.

Para construir novos programas podemos escrever diretamente códigos digitais que serão executados por um computador.

Uma maneira mais simples é usar um **compilador** para uma linguagem de programação específica.

Compiladores e LPs

Compiladores e Linguagens de Programação:

- Uma linguagem de programação é um conjunto de comandos que são mais “próximos” da linguagem humana do que os sinais digitais.
- Neste curso estamos interessados no estudo da linguagem de programação **Python**.
- Um **compilador** é um programa que lê um código de uma linguagem de programação e o transforma em um programa executável.
- O compilador realiza esta tarefa juntamente com um *assembler*.

Exemplo de código em diferentes níveis:

```
1 for i in range(10):  
2     c = a + b
```

Assembly:

```
loop:  add c, a, b  
       add i, i, 1  
       bnq i, 10, loop
```

Código binário:

```
01000010 00110101 01010100 0  
01100110 01110101 01010100 0  
11110000 01110101 01010100 0
```

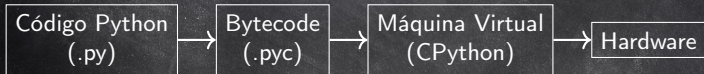
Python

- No caso específico de Python, o compilador traduz o código Python para o que chamamos de **bytecode**.
- O **bytecode** é um código de baixo nível que é executado em uma máquina virtual Python.
- Quando instalamos o compilador Python em um computador, na verdade instalamos tanto o **compilador**, o **interpretador** e a **máquina virtual** onde os programas em Python serão executados.
- O programa Python também é um **interpretador**, pois pode ser usado para executar diretamente os comandos em Python.

Como o Python é executado (CPython)

- O código Python é primeiro compilado em **bytecode** — um código intermediário.
- Esse bytecode é então interpretado por uma **máquina virtual**, fornecida pela implementação chamada **CPython** (escrita em C).
- O CPython funciona como:
 - ▶ Compilador (gera bytecode)
 - ▶ Interpretador (executa comandos)
 - ▶ Máquina virtual (executa bytecode)
- Isso torna o Python mais flexível e portátil, mas também **mais lento que linguagens como C**, que são compiladas diretamente para código de máquina nativo.

Comparação: Python (CPython) vs C



- **Python (CPython):** interpreta bytecode → execução mais lenta, porém mais flexível.
- **C:** gera código de máquina direto → execução mais rápida, mas menos portátil.

Sistema Operacional

- Os programas possuem instruções que são executadas no hardware.
- Mas o acesso ao hardware — como disco rígido, memória e processador — é controlado por um software especial conhecido como **sistema operacional** (SO).
- O sistema operacional é o responsável pelo controle do hardware, incluindo segurança, gerenciamento de memória, entre outros.
- Exemplos de sistemas operacionais: Windows, macOS, Linux, Android, iOS.

Algoritmos

- Ao criarmos um programa para realizar uma determinada tarefa, devemos ser capazes de construir algoritmos.
- **Algoritmo**: sequência de passos, precisos e bem definidos, para a realização de uma tarefa.
- Algoritmos podem ser especificados de várias formas, inclusive em português.
- **Exemplo de algoritmo**: como ordenar as cartas de um baralho?

Alguns algoritmos mais importantes da história

1. Algoritmo de Euclides

- ▶ Calcula o **MDC (máximo divisor comum)** entre dois inteiros.
- ▶ Um dos algoritmos mais antigos (século III a.C.), ainda usado em sistemas modernos.

2. Merge Sort (Ordenação por Intercalação)

- ▶ Proposto por John von Neumann em 1945.
- ▶ Utiliza **divisão e conquista** para ordenar dados de forma eficiente.

3. Transformada Rápida de Fourier (FFT)

- ▶ Converte sinais do domínio do tempo para o domínio da frequência.
- ▶ Essencial em **áudio, compressão, telecomunicações e imagem**.

4. Algoritmo de Dijkstra

- ▶ Resolve o problema do **caminho mais curto** em grafos ponderados.
- ▶ Base para sistemas de **GPS, redes e jogos**.

5. Criptografia RSA

- ▶ Base da **criptografia de chave pública**.
- ▶ Essencial para a segurança na **internet, assinaturas digitais, e-mails**.

Ambiente de desenvolvimento

- Acesso a um computador (ou dispositivo com acesso a internet).
- Para criar um programa, utilizamos um editor de texto (para escrever o código do programa) e um compilador/interpretador.
- O **compilador** é o que transforma o código em um programa executável.
- O **interpretador** é um programa que executa diretamente os comandos da linguagem.
- Se você usa Linux, macOS ou Windows, poderá utilizar qualquer editor simples como Emacs, Kyle, etc.
- Será preciso instalar o compilador/interpretador Python. Você pode baixá-lo do site (instale a versão Python 3.*):
 - ▶ <https://www.python.org/downloads/>
- O Python já vem com um editor chamado **IDLE**, que você poderá utilizar para escrever e rodar os seus programas, mas vamos utilizar um **IDE**.

Ambiente python

- O ambiente de desenvolvimento Python inclui:
 - ▶ Um amplo conjunto de Interfaces de Programação de Aplicações (APIs, do inglês “Application Programming Interfaces”)
 - ▶ Ferramentas de compilação e depuração de código
- Python 3 não é um Ambiente Integrado de Desenvolvimento (IDE, do inglês “Integrated Development Environment”)
 - ▶ A linguagem Python por si só não oferece editores de código ou ambiente de programação
 - ▶ Usaremos VScode (ou PyCharm) como IDE (editor + ambiente de programação) <https://code.visualstudio.com/download>
<https://www.jetbrains.com/community/education/#students>
<https://www.jetbrains.com/pycharm/download>

Exercício: Instale uma IDE Python. Clique aqui para uma breve Introdução IDE Python (2025)

Seu primeiro programa em Python

- Um programa em Python é um **arquivo de texto**, contendo declarações e operações da linguagem.
- Esse arquivo é chamado de **código fonte**.
- Por exemplo, o código abaixo exibe uma mensagem no terminal:

1

```
print("Olá turma!!")
```

- Você pode salvar esse arquivo como `hello.py`.
- Para executar este programa, abra um terminal e digite:

```
$ python3 hello.py  
Olá turma!!
```

Erros de código

- Caso o programa não esteja de acordo com as regras da linguagem, ocorrerão **erros de compilação** (ou erros de sintaxe, no caso do Python).
- Ler e interpretar essas mensagens de erro é uma habilidade fundamental para programadores.
- Veja o exemplo abaixo com erro de sintaxe:

```
1 print("Olá turma!!)
```

Execução no terminal:

```
$ python3 hello.py
File "<python-input-0>", line 1
  print("Ola turma!!)
    ^
```

SyntaxError: unterminated string literal (detected at line 1)

- O erro indica que a **aspas de fechamento está faltando**, o que causa um SyntaxError.

Tipos de Erros

- Erro de sintaxe
 - ▶ Falha na tradução do algoritmo para Python
 - ▶ O compilador/interpretador vai detectar e dar dicas
 - ▶ Mais fáceis de corrigir
- Erro de lógica
 - ▶ Resultados diferentes do esperado
 - ▶ Erro de projeto do algoritmo
 - ▶ Mais difíceis de detectar e corrigir

Validação de Programas

- **Teste de Mesa (ou "Chinês"):**
 - ▶ Validação manual do programa "em papel".
 - ▶ Acompanhe passo a passo a atualização das variáveis.
 - ▶ Você simula o papel do compilador e executor.
- **Juiz Remoto (Online Judge):**
 - ▶ Sistema web que executa o programa com entradas de teste.
 - ▶ Compara a saída do programa com respostas esperadas.
 - ▶ Verifica se o código funciona corretamente em todos os casos.
 - ▶ Usaremos o **URI Online Judge (Beecrowd)** e o **Rosalind**.
<https://judge.beecrowd.com/pt/>,
<https://rosalind.info/problems/locations/>